EngageIQ — Smart Engagement Opportunity Scorer

Shivneel Prasad · BAX-423 Big Data · Spring 2026 · UC Davis GSM

Live: https://engageiq-bax423-final.streamlit.app/ · Repo: https://github.com/shprasa/engageiq-bax423-final

1. Overview & local run

EngageIQ ranks GitHub engagement opportunities against interest profiles or four course personas. Streamlit dashboard: ranked cards, Why-this scores, Engage/Bookmark/Skip feedback, trend analytics, PDF/CSV export.

Grader local run (no API keys; offline data in ZIP). Unzip and copy/paste:

```
cd code
py -m pip install -r requirements.txt
py -m streamlit run app.py
```

Opens http://localhost:8501 (use python3 on macOS/Linux).

2. Data sources & offline dataset

Sources (≥2): GitHub REST API (scrape_github.py) + GitHub Archive (scrape_gharchive.py). Snapshot: 12,750 records (100% live API URLs), all 15 domains, in data/opportunities_snapshot.csv. DuckDB loads on startup. Built by scripts/build_snapshot.py.

Source	Total	Live	Notes
GitHub API	6,177	6,177	Repos, GFIs
GitHub Archive	11,842	11,842	Issue/PR events
Combined	18,109	18,109	100% live; 15 domains

3. System architecture

Python 3.11 · Streamlit · scikit-learn · DuckDB. Flow: scrape → CSV → stream/dedup → DuckDB → embed → rerank → UI.

Layer	Responsibility	Modules
Ingest	GitHub + GH Archive scrape → CSV; stream queue + URL dedup → DuckDB	scrape_*.py, streaming.py, data.py
Retrieve	TF-IDF/SVD embed; cosine ANN → top-200	embedding.py
Rank	Persona augment → 5-signal rerank → top-100	ranking.py
Learn	Thompson bandit over 15 domains	reinforcement_learning.py
Analyze	DuckDB batch SQL: volume, trends, WoW	analytics.py
Serve	Streamlit UI, cards, charts, PDF/CSV export	app.py, ui.py

4. Six core capabilities (all required — missing any caps score at 60/100)

#	Capability	Implementation
1	Multi-source ingest + streaming	GitHub API + GH Archive scrapers; OpportunityStream dedup; DuckDB; sidebar streaming controls
2	Embeddings + ANN retrieval	TF-IDF/SVD (128d) + NearestNeighbors cosine → top-200 candidates
3	Scoring + multi-stage ranking	Retrieve → augment → rerank (relevance, health, visibility, effort, recency); NDCG@10; Why-this chips
4	Adaptive learning / RL (50+ rounds)	Thompson bandit; Engage/Bookmark/Skip feedback; 60-round benchmark
5	Batch analytics + trends	DuckDB SQL: domain volume, daily trends, week-over-week rising domains
6	Dashboard + brief export	Streamlit tabs; ranked cards; suggested actions; PDF/CSV brief export

5. Pipeline design

Stage	Description
Retrieve	Embed profile + corpus (TF-IDF/SVD); ANN returns top-200 candidates
Augment	Inject persona-relevant GFIs, DevOps repos, archive events, devtools by keyword
Score	5 signals: relevance, health, visibility, effort, recency; persona-weighted; live URL +0.18
Rerank	RL domain weights + persona boosts → top-100 cards with component breakdown
Feedback	Engage/Bookmark/Skip updates bandit; shifts future domain mix
Analytics	DuckDB batch aggregates for trend charts; stream queue for ingest dedup

6. BAX-423 technique choices & rationale

EngageIQ integrates two BAX-423 techniques into the core ranking loop: a recommendation pipeline for retrieval and multi-stage ranking, and reinforcement learning for adaptive personalization from feedback.

Technique 1 — Recommendation system: profiles and opportunity text are embedded with TF-IDF (128-d SVD), indexed with cosine NearestNeighbors for ANN retrieval (top-200), then reranked on relevance, community health, visibility, effort, and recency with persona-specific boosts. TF-IDF/SVD was chosen over

deep embeddings because it runs fully offline in the ZIP, cold-starts for new profiles, and keeps Why-this scores interpretable. Ranking is benchmarked with NDCG@10 (simulated avg 1.00) and persona top-10 checks: Sofia 6/10 GFIs; David 10/10 DevOps; Raj 10/10 devtools; Lina 66% interest match.

Technique 2 — Reinforcement learning: Engage (+1.0), Bookmark (+0.85), and Skip (0.0) update a Thompson-sampling bandit over 15 domain arms; posterior samples shift domain weights in the reranker. This satisfies the 50+ feedback-round requirement without full deep RL. Over 60 simulated rounds, average reward in the last 10 improves from 0.23 to 1.00 (+0.78), showing the policy learns to deprioritize skipped domains (e.g., Raj low-engagement threads after simulated skips).

Benchmark	With RL	Without RL	Gain
Avg reward (last 10)	1.00	0.23	+0.78
Cumulative (60 rounds)	60	12.75	—

7. Test persona results (pass/fail table required)

Automated in persona_eval.py (exact PDF criteria). user_test_loop.py: 15/15 PASS.

Persona	Pass criteria (measured)	Result
Sofia	≥3 GFI (6/10) · no C++/Rust · ML threads · brief <1 hr (4/4)	PASS
David	K8s/infra (10/10) · niche repos · discussions (3/3)	PASS
Lina	Recency/velocity · WoW analytics · rising domains (3/3)	PASS
Raj	DevTools (10/10) · threads · RL skip learning (3/3)	PASS

Six capabilities × four personas (24/24 PASS):

Capability	Sofia	David	Lina	Raj
1 Ingest + streaming	PASS	PASS	PASS	PASS
2 Embeddings + ANN	PASS	PASS	PASS	PASS
3 Scoring + ranking	PASS	PASS	PASS	PASS
4 RL (50+ rounds)	PASS	PASS	PASS	PASS
5 Batch analytics	PASS	PASS	PASS	PASS
6 Dashboard + export	PASS	PASS	PASS	PASS

8. Limitations

- Two sources only (GitHub API + GH Archive); GH Archive threads proxy Reddit/blog discussion items in persona validation.
- In-process streaming + DuckDB dedup, not Kafka/Spark. TF-IDF/SVD vs deep embeddings; hidden personas not pre-tested.
- Suggested actions use offline templates unless optional LLM API keys configured.

9. Submission & deployment

ZIP: Prasad_Shivneel_BAX423_Final.zip (code/, data/, brief.pdf, prompts.md). Deploy: Streamlit Cloud → https://engageiq-bax423-final.streamlit.app/, entry code/app.py. Demo: Sofia GFIs → Engage/Skip RL → David DevOps → Lina WoW chart → Raj devtools → export brief.